

Gateway API doesn't solve real problems - yet

Disclaimer: This tells my personal experiences with the Kubernetes Gateway-API and shows the value Ingress-Nginx provided. The disconnect between complexity and value Gateway-API provides shows a fundamental problem cooperations face open-source!

My takeaway: Gateway API makes hard things possible - but easy things hard.

Why the Change?

The Kubernetes steering committee shocked a lot of people when they announced the [retirement of the ingress-nginx-controller](#) at the 2025 Atlanta-Kubecon.

After March 2026, the biggest ingress controller will not receive any more updates and patches. This change resulted in quite some angry comments towards the maintainers and the committee. But this change was foreseeable. The maintainers repeatedly reached out for more support or asked for corporate sponsors - which were unanswered.

The decision is understandable, at least. A free, feature rich reverse proxy with good documentation that is used by thousands for free. Some corporations have generated billions in sales with requests that were served by [ingress-nginx](#), but the only thing that was given back were support requests, bug reports or new feature requests.

I will not go into the debate of corporations exploiting open-source projects - this has been talked about a lot recently. The decision has been made, and the direction given was clear. The future is the [Gateway-API](#)!

Why a new API?

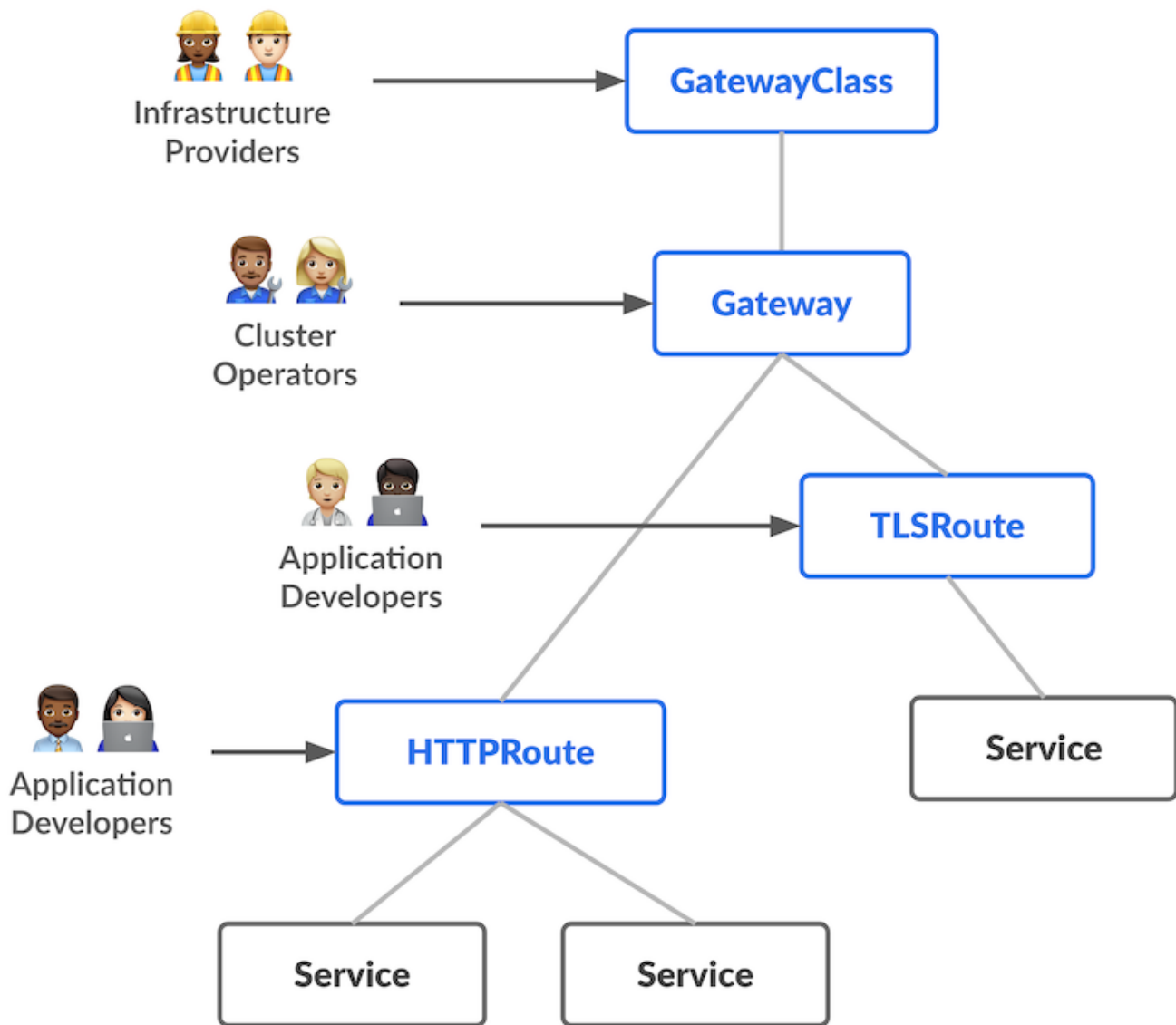
The [Ingress](#) API is a core component of the [networking.k8s.io](#) Kubernetes API-Group. It defines the routing path for HTTP-Traffic (L7-Layer) via a bunch of rules with [PATH](#) and [Host](#). It can also do TLS for these rules but that's pretty much it.

It does not handle headers like [X-Forwarded](#), cookies, authentication, payload size, compression, timeouts, redirects, traffic splitting and many more. All of these things **can** be done with ingress though, but not via its core API-Definition.

Annotations are commonly used to enrich API-Objects with additional information. Ingress-Nginx had over 110 of them. A problem that comes with Annotations is that they are not validated, since they are not part of the API-Spec and that they are not vendor agnostic. So you can't use same annotations from Ingress-Nginx for Kong, HAProxy, Traefik or the Apache-Webserver-Ingress. Each of these have their own feature- and annotation set, switching ingress-controller usually takes some time.

Another concern is that all these settings are applied at the ingress level. Should a developer with [Ingress](#) access really be able to alter the TLS config for a whole domain?

Gateway-API promises to fix this. A well-defined API that separates operational tasks like TLS and proxy settings from routing configuration.



Gateway-API introduces a stricter role-based approach. It has a mental model of *how does what* and which resources can be referenced from where. It's **HTTPRoute** supports native routing via hostname, header, path or query parameters. All well defined in its core API but still extendable via filters.

Apart from HTTP routing, Gateway-API also proposes **GRPCRoutes**, **TLSRoutes**, **TCPRoutes** and **UDPRoutes**. While some of these are still experimental, having a well-defined, vendor-agnostic spec for all these use-cases is a killer feature.

How does this affect Developers and Admins?

In theory this means less error-prone routing configuration (let's see how this will work out) and developers not having to worry about TLS.

In practice, developers will need to learn a new doc-spec or wait for admins to give them templates.

Admins on the other hand will have a lot more to do. Configure the Gateway, provide templates for developers and allow all necessary namespaces to reference the Gateway. They need to test and set up new integration, handle TLS and maintain a separate API that is maintained outside the Kubernetes core API.

Gateway API is more complex

With more features comes more complexity.

The Ingress API has two resource types, `Ingress` and the hardly noticed `IngressClass` type. The Gateway-API has `ReferenceGrant`, `BackendTLSPolicy`, `GRPCRoute`, `Gateway`, `GatewayClass`, `HTTPRoute` and these are just the stable API-Objects, there are a lot more in experimental mode.

Ingress is a core component of Kubernetes, the Gateway-API is not shipped with Kubernetes and needs to be installed separately via `CRDs`. This might result in bootstrapping challenges since one cannot expect that the APIs are installed.

```
To install them use kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/<LATEST_VERSION>/standard-install.yaml. The --server-side is required since the API is too big for client side apply.
```

To setup a `Gateway` cluster admins need to install `CRDs`, choose an implementation, install an operator/controller and then create the `Gateway`. Most Gateway-API implementations separate the controlplane from the dataplane. Most Ingress implementations used a shared approach where an agent ran as a sidecar process next to the dataplane to ensure the proper configuration.

Right now, the selecting a Gateway is actually the hardest part. While the main API-Objects specs have reached GA, their features are not yet supported by all implementations. While the Gateway-API SIG tries to help with this decision by providing [comparison tables](#) and [conformance reports](#) it is still a lot of work to go through.

These on-paper comparisons don't replace real-world tests to catch any edge cases and say nothing about performance between all these. If you look for some real-world comparisons, you may wanna check out [this benchmark](#) from [John Howard](#).

Gateway API is not vendor agnostic

Lets remember the goals of Gateway API: Separate personas, strict validation and a vendor-agnostic config.

Right now I see a trend that is actually moving against the last goal. Yes, TLS configuration and routing might be standardized, but the `nginx-gateway-fabric` for example already brings their own set of 10 `CRDs` for configuring the `Gateway` itself or for use cases that are not yet defined in the standard Gateway-API spec. And so does Kong and Envoy-Gateway with even more `CRDs`.

There is now native way to do auth or mTLS enforcement (for clients) in the Gateway-API yet, so implementations bring their own filters and `CRDs`.

Gateway API, by design, is extendable and vendors are using this extendability to overcome the shortcomings of the core API. They are using these to separate themselves from each other and maybe - to create a vendor-lock-in. Never forget that there are companies behind these products that want and need to make money.

Maybe over time more features will be covered by the core Gateway-API, but most of us know even if the spec is well defined, implementations can behave differently. My `TLSRoute` worked perfectly with Cilium but was not working with `nginx-gateway-fabric`.

Gateway API is not widely supported yet

New technologies need time for adoption, it is totally normal that most helm charts do not yet ship with `HTTPRoutes` alongside `Ingress`. Unfortunately a lot of helm charts do not support a `extraObjects` either, so you need multiple commands to deploy your app or a multi source GitOps solution.

Creating a general helm template for `HTTPRoutes` is actually quite hard. I know this because I crated the `HttpRoute` helm template that gets generated when you run `helm create <my-chart>` in recent versions of helm.

I wanna give another example of a problem I encountered:

A common scenario in highly automated clusters is having an Ingress-Controller, CertManager and ExternalDNS. These work perfectly together. The ingress is created and uses a fallback TLS certificate, ExternalDNS checks the domain records and adds a new record if not set yet and Certmanager creates a `http-01-challenge` to obtain a valid certificate. It may take some minutes, but *eventually* you have a valid, working `Ingress` with TLS.

Now try the same with Gateway-API. It will not work, even with experimental flags. Why does it not work? The Gateway handles the TLS, not the `HTTPRoute`. The default listener likely does not have a valid certificate for your new domain, so you add a new listener. The `HTTPRoute` references that listener, but never becomes ready because the listener is not healthy yet due to the missing certificate. ExternalDNS watches `HTTPRoutes`, but only the ones that are ready and have an IP. Now the circle closes, without a valid DNS record, you can't complete a `http-01-challenge`. It's a deadlock.

The only way around this that I found was to switch to an `dns-01 challenge`. The ExternalDNS project is aware of this problem, but AFAIK the solution is still work-in-progress.

Ingress-Nginx will be missed

I don't want to make Gateway-API look bad; it is powerful - maybe not just ready yet.

For those not ready to switch: Maybe look at Cainguards `EmeritOSS` program. They `announced` to provide basic security maintenance for `ingress-nginx`.

For these reason, Ingress-Nginx will be missed. It is a dead simple solution that just works. The battle-tested Nginx with a small agent that does config generation and reloads. Provided by a couple of individuals that did get nothing in return but provided the technology that served half of all Kubernetes-hosted HTTP traffic. Ingress-Nginx is a community project - no big company behind it. That's something that will change with Gateway-API. Implementing all of Gateway-APIs features is a lot of work, no one can expect a couple of individuals to do that in their free time.

Now it's done by corporations that battle for the market share that Ingress-Nginx will leave behind. Once this battle is settled, I wouldn't be surprised to see more Broadcom, HashiCorp and Docker like rug-pulls.